

Le patron de conception Singleton

Durée : 45 min

Crédit : <https://www.math.univ-paris13.fr/~chaussar/>

1 Instructions générales

1. Téléchargez l'archive zip associée à cet énoncé sur la plateforme pédagogique.
2. Ouvrez le projet dans l'IDE IntelliJ IDEA (ou autre).
3. Si vous ne l'avez pas encore fait, configurez votre IDE pour n'accepter que des attributs avec un identifiant commençant pas un '_'. (*Dans IntelliJ IDEA, menu Settings::Editor::Code Style::Java, ajoutez le caractère '_' dans field et static field. Puis, dans le menu Settings::Editor::inspection::Java::Naming Conventions ajoutez la vérification de Field naming convention (instance field et static field) avec l'expression régulière : `_[a-z][A-Za-z\d]*`.*)
4. Exécutez le programme pour vérifier que tout fonctionne correctement.

2 Données de l'exercice

Il s'agit d'un exercice simple : des avions sont créés par le programme principal. L'aéroport possède une tour de contrôle (classe `ControlTower`), qui gère l'unique piste. Quand un avion (classe `Plane`) souhaite décoller ou atterrir, il attend que la tour de contrôle le lui permette. Deux avions ne doivent pas réserver la piste en même temps, sous peine d'un accident.

La classe `LoadAndShow` ne vous intéresse pas, pas besoin de la regarder.

3 Exercice 1

Si vous exécutez le programme, vous devez terminer sur un accident. Dans la fonction `main()` de la classe `Main`, l'appel de la méthode `start()` sur un `Thread Plane` appelle sa méthode `run()` qui crée une nouvelle tour de contrôle `ControlTower` à chaque fois. Ce n'est évidemment pas souhaitable.

Modifiez la classe `ControlTower` pour en faire un Singleton, afin que les `Plane` se connectent tous à la même tour de contrôle : une variable de classe de type

`ControlTower`, le constructeur devient privé et une fonction statique `getInstance()` crée un objet de type `ControlTower` seulement s'il n'en existe pas.

Cette fois, le programme doit se terminer sur un succès.

4 Exercice 2

Maintenant, vérifions si le singleton `ControlTower` est robuste à la création de threads. On va imaginer que la création d'un objet `ControlTower` prend du temps. Dans la méthode `run()` de la classe `Plane` remplacez l'instruction :

```
FactoryControlTower f = new FactoryControlTower();
```

par les deux lignes suivantes :

```
Thread.sleep(500);  
FactoryControlTower f = new FactoryControlTower(); :
```

et placez tout le code la méthode dans un `try catch`.

Cela endormira le thread `Plane` qui tente de créer un `ControlTower` pendant 500 ms. Testez votre code : que se passe-t-il ?

Il faut faire en sorte que, si plusieurs threads `Plane` tentent d'obtenir un `ControlTower`, un seul puisse créer le `ControlTower` à chaque fois. Comment mettre en place cette solution (regardez le mot clef `synchronized` en Java).

5 Conclusion

Vous venez d'améliorer votre patron Singleton afin qu'il prenne en charge une exécution multi-thread. Le patron Singleton est souvent utilisé dans le cas de threads afin de créer un seul objet centralisé sur lesquels certains threads se connectent, d'où l'importance de la dernière partie de l'exercice que vous venez de mettre en place.